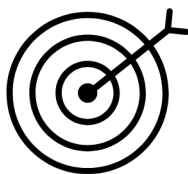




AULA 2

Redes neurais profundas: arquiteturas avançadas



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Entender para que servem as redes neurais recorrentes (RNNs) e como aplicar as *long short term memory networks* (LSTMs).
- Apresentar problemas que utilizam redes *autoencoders*.
- Discutir os principais conceitos das *generative adversarial networks* (GANs).

Que história é essa?

Você viu, na aula anterior, como funcionam as redes neurais artificiais e suas principais características. Entendeu como pequenas unidades simples (chamadas *perceptrons*) podem ser agrupadas para resolver problemas complexos. Também conheceu as redes neurais profundas e suas novas organizações em camadas para resolver problemas ainda mais robustos.

Nesta aula, você vai aprender outras arquiteturas de redes neurais profundas e entender como e quando devem ser aplicadas.

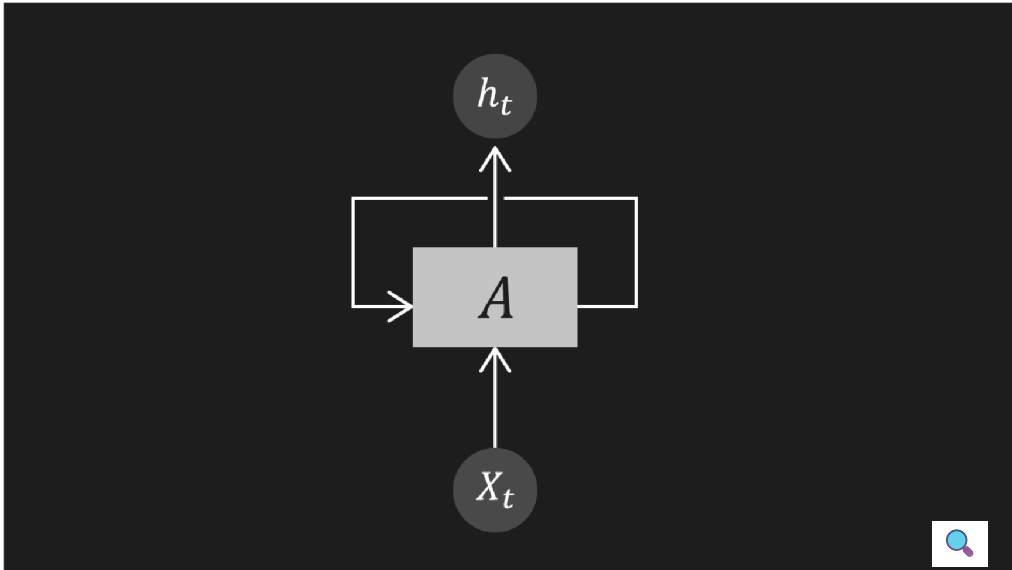
Redes neurais recorrentes

Interativo

No interativo, vemos uma caixa azul em um fundo branco, com o texto: ‘Você se lembra das redes neurais feedforward?’. Acima do texto, vê-se uma figura com pontos de interrogação dentro de caixas de fala, como as usadas em quadrinhos. Abaixo da caixa, lê-se: clique no card para saber mais. Ao clicar, vemos o seguinte texto: Nelas, os pesos sinápticos são congelados após o treinamento e não se modificam.

As saídas dependem apenas da amostra de entrada e do processamento feito pelos *perceptrons*. Elas são adequadas à maioria dos problemas em que a entrada é pontualmente processada pela rede, resultando em uma saída. Existem problemas em que o processamento do sinal deve considerar o histórico dos valores de entrada.

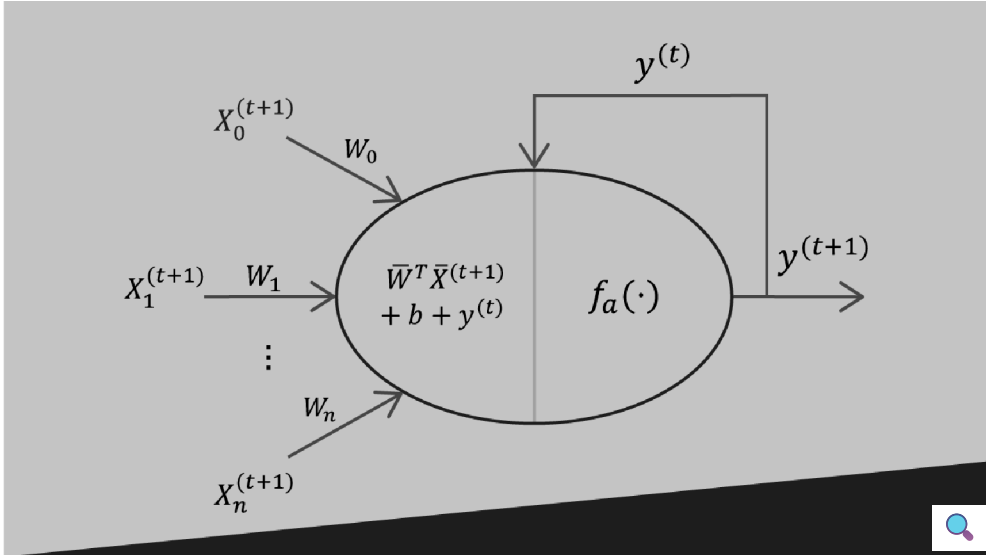
Veja as séries temporais: cada dado é ordenado em função do tempo e sofre influência de eventos anteriores. Exemplos: temperatura, áudio, vídeo, etc.



Nesses casos, o processamento do tempo t deve considerar a tendência ou, ao menos, a influência do passado recente em $t-1$. Foi justamente pensando nesses casos que se criaram as **redes neurais recorrentes (RNNs)**.

As RNNs constituem uma ampla classe de redes, cuja evolução do estado depende tanto da entrada corrente quanto do estado atual.

Nas RNNs, o neurônio não é mais uma unidade computacional de alimentação direta. A conexão de *feedback* o força a lembrar o passado e usá-lo para prever novos valores.



$$\begin{cases} y^{(t+1)} = f_a(\bar{w}^T \bar{x}^{(t+1)} + b + y^{(t)}) \text{ para } t > 0 \\ y^{(0)} = 0 \end{cases}$$

Nos *perceptrons* das RNNs, existe uma sinapse que recebe a previsão da rodada anterior e a adiciona à nova saída linear. O valor resultante é transformado pela função de ativação, para produzir a nova saída real. Convencionalmente, a primeira saída é nula, mas isso não é uma restrição.



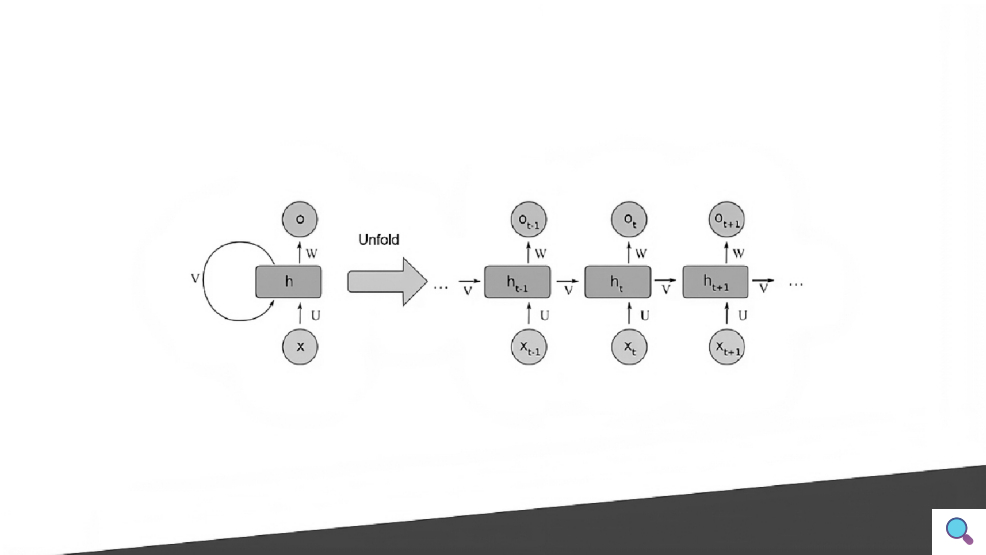
Fique ligado

Por ser um sistema dinâmico, deve-se tomar cuidado com funções de ativação que saturam o sinal (como Sigmoid e Tangente Hiperbólica).

Desdobramento no tempo (*unfolding in time*)

Já que **os dados** que você aprenderá a processar **são séries temporais**, as **entradas da RNN são janelas de tempo** que serão relacionadas com a saída desejada. A entrada que recebe essas janelas se chama desdobramento no tempo e é uma sequência de valores, um em cada passo de tempo.

Você pode entender uma RNN que recebe uma sequência de vetores de entrada como uma rede feedforward cujas matrizes de pesos são compartilhadas a cada passo de tempo **t**. Onde uma entrada **x** é afetada por uma matriz de pesos **U**, um estado **h** é afetado por uma matriz **V** e a saída **o** (que será a entrada da próxima camada da rede) é afetada por uma matriz **W**.



O desdobramento no tempo permite o aprendizado de mapeamentos entre sequências de entrada e saída com diferentes tamanhos, clique nas flechas para explorar o assunto:

Interativo

Interativo que, no primeiro slide, vê-se um retângulo cinza denominado One to many. Nele, vê-se três fileira de retângulos: acima, três azuis, no centro, três amarelos, e embaixo, um vermelho. O vermelho é conectado aos amarelos por uma seta vermelha; os amarelos são conectados entre si por setas amarelas, e aos retângulos azuis, por setas azuis. Ao lado, lê-se: Um-para-muitos. Exemplos: criar legendas para uma imagem, descrever objetos ou criar um parágrafo contando o que há na imagem. No segundo slide, vê-se um retângulo cinza denominado Many to one. Nele, vê-se três fileira de retângulos: acima, um azul, no centro, três amarelos, e embaixo, três vermelhos. Os vermelhos são conectados aos amarelos por setas vermelhas; os amarelos são conectados entre si por setas amarelas, e ao retângulo azul, por uma seta azul. Ao lado, lê-se: Muitos-para-um. Exemplos: análise de sentimento (positivo ou negativo) em uma sentença em português e classificar um texto por assunto. No terceiro slide, vê-se dois retângulos cinzas, ambos denominados many to one. O primeiro apresenta três fileiras de retângulos: acima, três azuis, no centro, cinco amarelos, e embaixo, três vermelhos. Os vermelhos são conectados aos três dos amarelos por setas vermelhas; os amarelos são conectados entre si por setas amarelas, três deles são conectados aos retângulos azuis, por setas azuis. O retângulo da direita apresenta três fileiras de retângulos: acima, três azuis, no centro, três amarelos, e embaixo, três vermelhos. Os vermelhos são conectados aos amarelos por setas vermelhas; os amarelos são conectados entre si por setas amarelas, e aos retângulos azuis, por setas azuis. Ao lado, lê-se: Muitos-para-muitos. Exemplos: tradução de frases do inglês para o português, rotular os quadros (frames) de um vídeo e converter áudio falado em texto (text-to-speech).

Rede LSTM

A rede LSTM (*long short term memory*) é análoga ao comportamento das memórias de curto e longo prazo cerebrais.



Sua principal ideia é manter uma memória de curto prazo que define o contexto e a memória de trabalho, enquanto a memória de longo prazo define o aprendizado do conhecimento adquirido.

Foi proposta em 1997, por Sepp Hochreiter e Jürgen Schmidhuber, e, apesar dos anos, continua sendo o estado da arte em redes recorrentes.

A LSTM é adequada quando há intervalos muito longos de tamanho desconhecido entre eventos importantes.



Fique ligado

Durante o treinamento, uma rede LSTM pode aprender quando deve manter ou descartar informações.

Em comparação, as redes recorrentes clássicas são focadas no aprendizado, mas não no esquecimento seletivo.

Dessa forma, a **LSTM** pode **otimizar a memória**, para lembrar o que é realmente importante, e remover todas as informações que não são necessárias, para prever novos valores.

Para realizar suas tarefas, a LSTM propõe explorar dois recursos importantes: estado e portões.

Estado

↓

É um conjunto separado de variáveis que armazenam os elementos necessários para construir dependências de longo e curto prazo. Inclui as informações do estado corrente e as variáveis responsáveis pelo gerenciamento cíclico e interno do erro fornecido pelo algoritmo de retropropagação.

Portão

↓

Atua como um elemento que pode modular a quantidade de informação que flui por ele. Por exemplo, você pode supor que a equação $y = ax$ define um portão. A variável $a \in [0, 1]$ pode ser considerada o estado desse portão. Quando é igual a 0, o portão bloqueia a entrada x , e quando é igual a 1, permite que a entrada flua sem restrições.

Qualquer valor intermediário reduz a quantidade de informação proporcionalmente, mas permite que a informação passe por esse

canal.

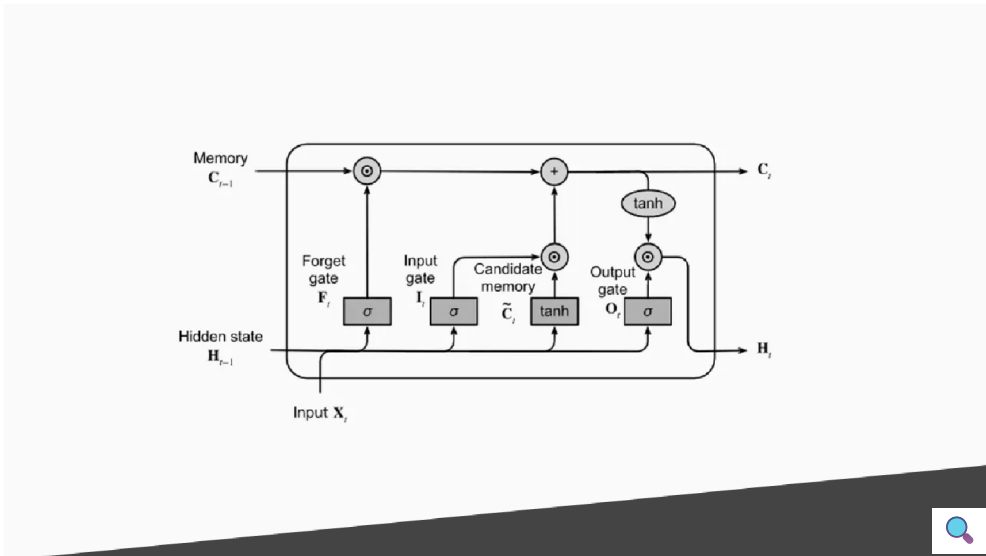


Fique ligado

Note que os portões são gerenciados por funções sigmoides, enquanto as ativações são baseadas em tangentes hiperbólicas, cuja simetria garante melhores desempenhos.

Célula LSTM

Uma célula LSTM é um tipo especial de unidade recorrente usada em redes neurais para processar e aprender a partir de sequências de dados, como um texto ou séries temporais. Ela foi projetada para permitir que a rede memorize informações de longo e curto prazo em seus estados internos.



Uma célula LSTM tem como entrada e saída os seguintes elementos:

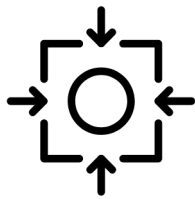
- **Entrada (X_t):** A entrada atual X_t é o novo dado que a célula recebe em cada passo de tempo, como uma palavra em uma frase ou um valor em uma série temporal.
- **Estado Oculto (H):** O estado oculto, mostrado como H_{t-1} (anterior) e H_t (atual), carrega informações de curto prazo e é a saída da célula que será passada para a próxima etapa ou camada.
- **Memória (C):** A memória, representada por C_{t-1} (memória anterior) e C_t (memória atual), é o núcleo da LSTM. Ela armazena informações de longo prazo que podem ser atualizadas ou descartadas.

Portões

Os portões controlam o fluxo de informações na célula usando funções de ativação, geralmente a função sigmoide (σ), que gera valores entre 0 e 1, indicando quanto se deve passar ou bloquear dessas informações. Os portões da célula LSTM são categorizados da seguinte forma:

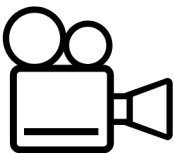
- **Portão de Esquecimento (F_t):** Decide que informações da memória anterior C_{t-1} devem ser descartadas. Um valor próximo de 0 significa "esquecer", e 1 significa "manter".
- **Portão de Entrada (I_t):** Determina quais novas informações da entrada X_t e do estado oculto H_{t-1} serão adicionadas à memória.

- Portão de Saída (O_t): Controla quais partes da memória C_t serão usadas para gerar o novo estado oculto H_t . Para isso ele usa uma memória candidata ($\hat{C}_t$) usando a função $\tanh$, que produz valores entre -1 e 1, combinando X_t e H_{t-1} .



Mantendo o foco

O portão de esquecimento expressa a probabilidade de a última sequência ser mais importante que o estado atual. Já o portão de saída expressa a probabilidade de a sequência atual ser capaz de deixar o estado atual fluir. Apenas o portão de entrada $i^{(t+1)}$ tem a responsabilidade de conceder a ela o direito de influenciar o novo estado.

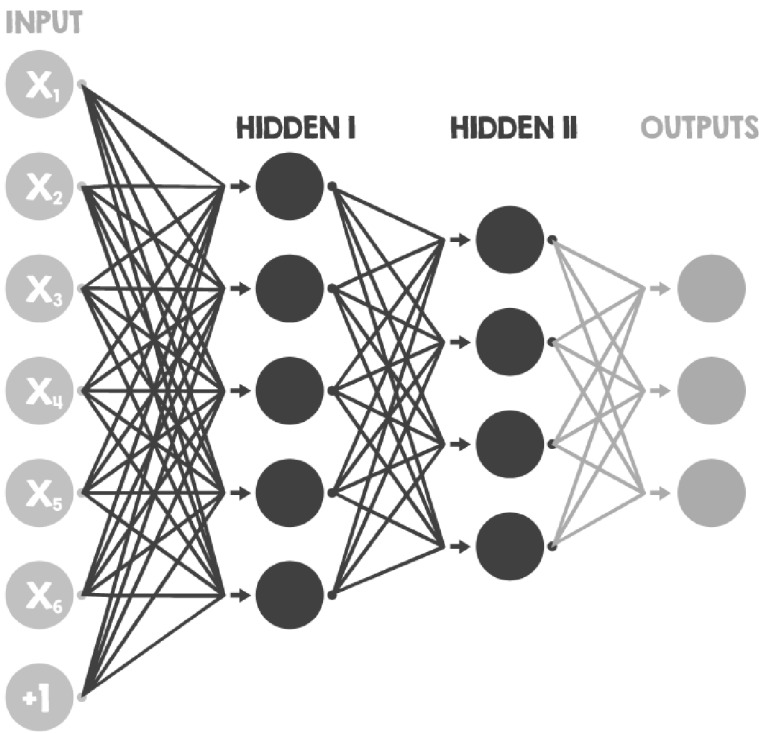


Arquiteturas profundas

Neste vídeo você vai conhecer um pouco mais sobre arquiteturas profundas, juntamente com alguns exemplos.



Autoencoders



Os *autoencoders* são arquiteturas que oferecem uma **abordagem diferente para problemas clássicos** de redução de dimensionalidade ou aprendizado de dicionários.

Por serem adaptativos, não sofrem as limitações que afetam modelos estatísticos clássicos. Você pode utilizá-los para explorar camadas neurais específicas e extrair informações dos dados, definindo critérios específicos (por exemplo, características de rostos de pessoas no *dataset*).



Fique ligado

A rede cria representações internas que podem ser mais robustas a diferentes tipos de distorções e muito mais eficientes, por permitirem uma maior quantidade de informações a serem processadas.

Sabe-se que, em um conjunto de dados, cada coluna representa uma dimensão em um espaço vetorial complexo. Muitas vezes, as amostras são representadas em uma alta dimensão, mas também podem ser expressas em uma dimensão menor, sem perda de informação.

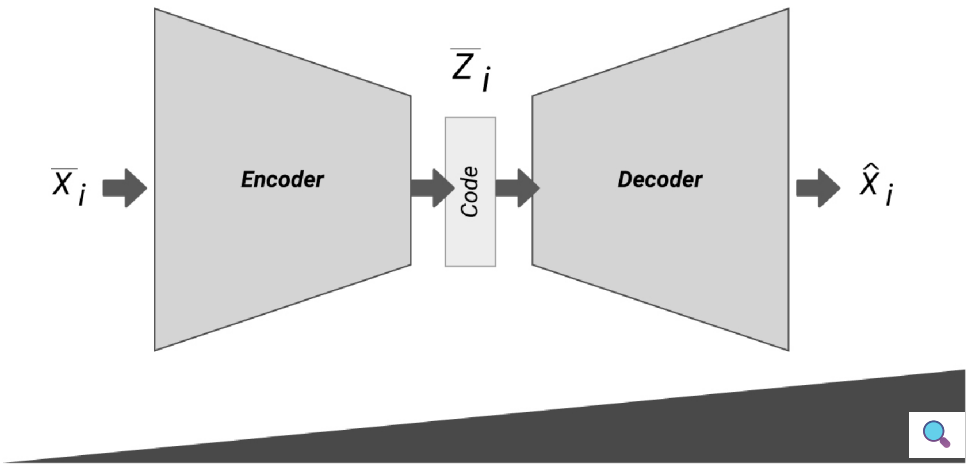
Esse conceito é importante, pois a complexidade de um modelo é proporcional à dimensionalidade dos dados de entrada. Existem muitas técnicas estatísticas para tentar reduzir o número real de componentes válidos. Na próxima aula, você vai entender mais sobre elas.

Um *autoencoder* genérico é um modelo dividido em dois componentes separados: *encoder* e *decoder*. Eles não funcionam completamente autônomos e são treinados juntos.

Interativo

O interativo mostra duas caixas azuis em um fundo amarelo. Na primeira, lê-se ‘codificador’, e na segunda, ‘decodificador’. Ambas são ilustradas por uma figura de nós (círculos brancos) conectados por linhas. Acima, lê-se: clique nos cards para saber mais. Ao clicar nas caixas, lemos: Codificador: Sua tarefa é transformar uma amostra de entrada em um vetor de características codificado. É como se ele aprendesse a descrever, em poucas dimensões, cada uma das amostras. Decodificador: Deve reconstruir a amostra original, usando o vetor de características como entrada. Isso significa que ele aprende a recriar o dado/imagem original a partir da descrição acordada com o codificador.

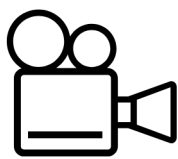
Observe a imagem a seguir:



Pode-se definir o codificador como uma função onde a saída Z_i é um “código” vetorial de dimensionalidade bem menor que as entradas. Onde X é o conjunto de amostras contidas nos dados de treinamento. Da mesma forma, pode-se entender o decodificador como outra função, em que a saída é um vetor reconstruído, de dimensionalidade igual ao vetor original.

Durante o aprendizado, uma rede *autoencoder* é vista como um par de redes acopladas: uma codificadora e a outra decodificadora. Pode-se entender que a rede codificadora realiza a “compressão” dos dados de entrada e a rede decodificadora, a “descompressão” para a sua forma original.

A representação codificada, muitas vezes, é chamada de representação latente, pois cada dimensão pode representar uma característica da amostra original.



Exemplos de *autoencoders*

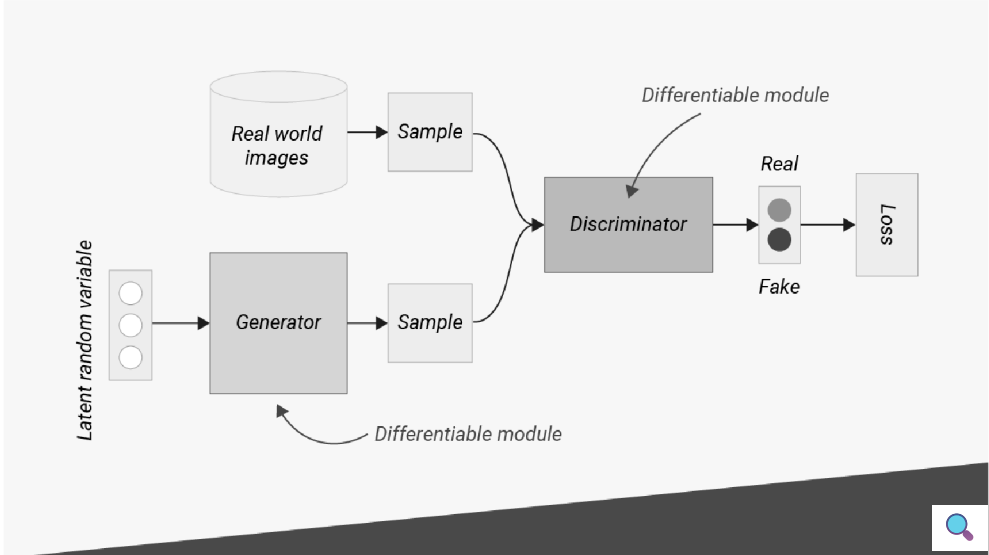
Neste vídeo você terá a oportunidade de retomar sobre os *autoencoders*, verá exemplo desta aplicação, e por fim, verá também exemplos de aplicação de *denoising autoencoders* e *variational autoencoders*.



GANs (redes adversárias generativas)

A GAN é uma arquitetura composta de duas redes neurais que **competem entre si**.

A primeira é uma rede geradora, que deve ser capaz de gerar/criar informações o mais fiéis possível às características das amostras usadas durante o treinamento. A outra rede deve aprender a criticar suas entradas e decidir se os dados têm características que os tornem verdadeiros ou se seriam falsos e gerados artificialmente.



A ideia por trás dessa organização é que uma rede motiva o treinamento da outra e ambas avançam no treinamento dos dados. Por exemplo, uma GAN treinada em fotografias pode gerar novas imagens que parecem autênticas (ao menos superficialmente) para observadores humanos, apresentando muitas características realistas.

- 1 Gerador: gera amostras falsas, tenta enganar o discriminador.
- 2 Discriminador tenta distinguir entre amostras reais e falsas.
- 3 Treinam-se uns contra os outros.
- 4 Após repetições, obtêm-se um gerador e um discriminador melhores.

Processo de treinamento

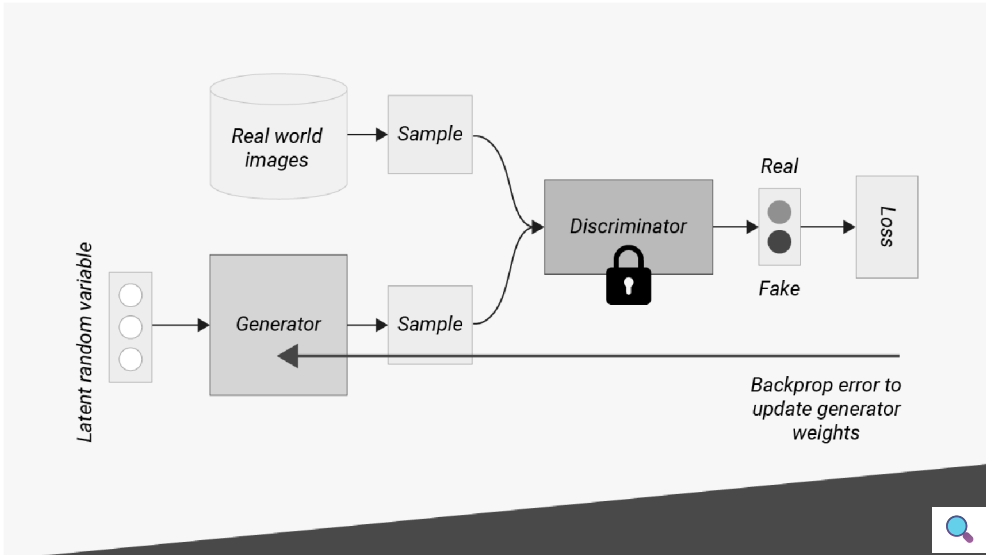
O treinamento contraditório (*adversarial training*) visa **aprender a distinguir** entre amostras verdadeiras e falsas, fazendo com que o componente que gera amostras se aproxime cada vez mais dos exemplos de treinamento.



Fique ligado

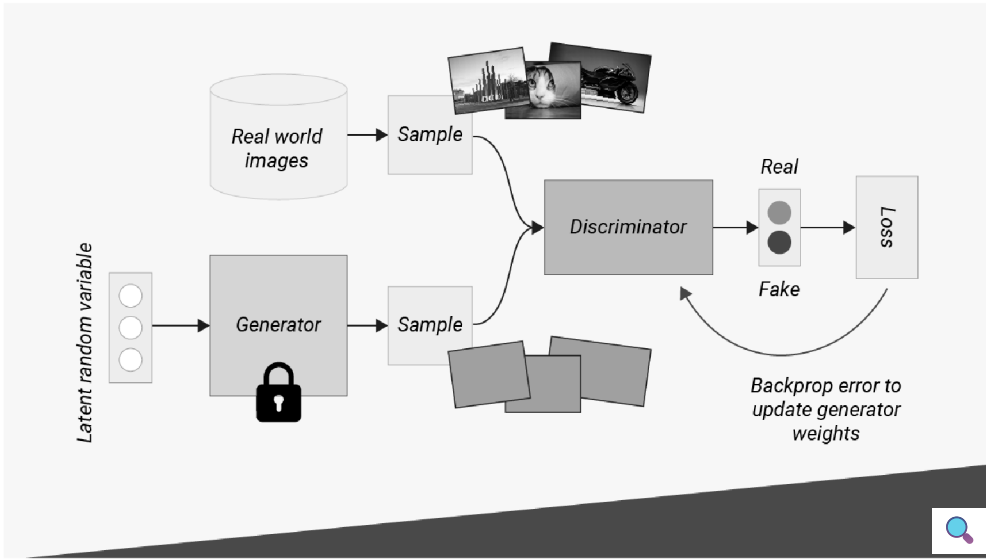
A ideia de treinamento contraditório é uma abordagem parcialmente inspirada na teoria dos jogos (algoritmo *minmax* para jogos competitivos).

Na prática, o gerador deve criar uma amostra *x*, que deverá ser avaliada pelo discriminador. Este, deve calcular a probabilidade de os dados do vetor *x* serem verdadeiros. Se *x* realmente pertencer ao conjunto de dados treinados, o discriminador produz um valor próximo a 1. Caso seja muito diferente das outras amostras verdadeiras, o discriminador produzirá uma probabilidade muito baixa como saída.



Nessa etapa, a ideia do jogo de competição é criar um ambiente para treinar o gerador para enganar o discriminador, produzindo amostras que potencialmente podem ser extraídas do conjunto de dados reais.

Em seguida, quando o gerador é bom o suficiente para enganar o discriminador, é sua vez de treinar para melhorar sua avaliação.



Após repetições, obtêm-se um gerador e um discriminador melhores. A primeira arquitetura da família GAN proposta foi a Deep Convolutional Generative Adversarial Network, fortemente influenciada pelas arquiteturas que você viu anteriormente.

Deep Convolutional GANs (DCGANs)

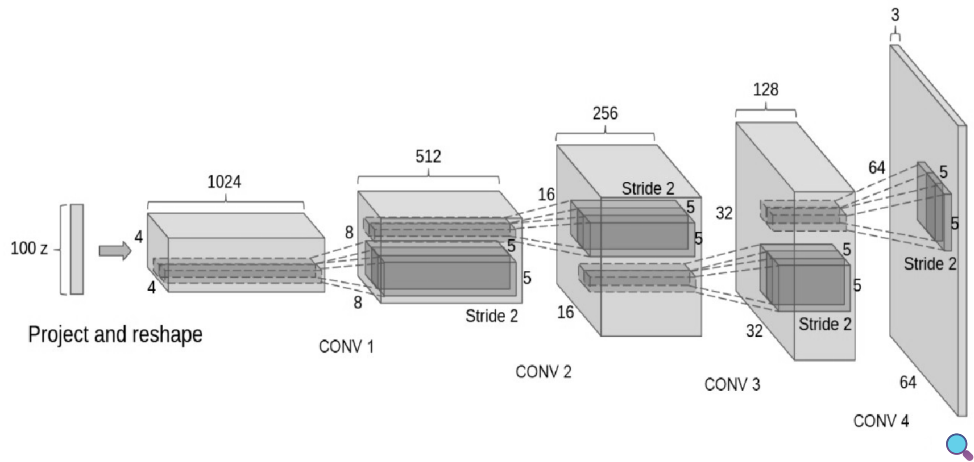
As DCGANs foram propostas no artigo “*Unsupervised representation learning with deep convolutional generative adversarial networks*” (RADFORD; METZ; CHINTALA, 2016), e seu gerador tem quatro

convoluções de transposição, com tamanhos de kernel iguais a (4, 4) e stride (2, 2).

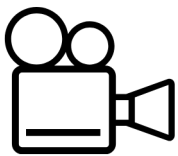


Fique ligado

A entrada é um único *pixel* multicanal ($1 \times 1 \times code_length$), expandido por convoluções subsequentes.



Os autores ainda sugeriram o uso de um conjunto de dados de valor simétrico e, por isso, normalizam-se os valores entre -1 e 1 em lote após cada camada convolucional. Também se utiliza a função de ativação Leaky ReLU (com *slope* 0,2).



Redes adversárias generativas

Neste vídeo você irá conhecer um pouco mais sobre as redes adversárias generativas e seu processo de treinamento.



Refleta

Agora que você viu as principais arquiteturas da atualidade, reflita sobre dois pontos:

- As redes neurais são tão mágicas como a mídia vem noticiando?
- Quais são as possíveis aplicações em que as redes são mais adequadas para uso?



Referências

Clique aqui para acessar as referências e créditos desta aula.



BONACCORSO, G. **Machine learning algorithms**. 2. ed.
Birmingham: Packt Publishing, 2018.

BONACCORSO, G. **Mastering machine learning algorithms**.
Birmingham: Packt Publishing, 2018.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep learning**.
Cambridge, MA: MIT Press, 2016.

OLAH, C. Deep learning, nlp, and representations. **GitHub blog**,
2015. Disponível em: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>. Acesso em: 23 mar. 2023.

RADFORD, A.; METZ, L.; CHINTALA, S. Unsupervised
representation learning with deep convolutional generative
adversarial networks. 2015. Disponível em:
<https://doi.org/10.48550/arXiv.1511.06434>. Acesso em: 24
mar. 2023.

Banco de imagens Pexels, Pixabay, Envato, Freepik e
Shutterstock.